Tasks on Terraform-04

1. Watch Terraform-04 video.

Watching video of terraform-04 completed.

2.Execute the Terraform-04 Script

Implementation steps:

1. Watched the Terraform-04 tutorial video.
2. Created a directory for the Terraform project.
3. Created the Terraform configuration files as shown in the video.
4. Configured the required provider and resources in the Terraform script.
5. Opened the terminal in the project directory.
6. Executed the Terraform configuration using terraform apply.
7. Verified that the resources were created successfully.

Example:

=======

```
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
      version = "2.3.0"
    }
  }
}


resource "local_file" "my-pet" {
filename = "pets.txt"
content = "I love cats!"
}
```

=========

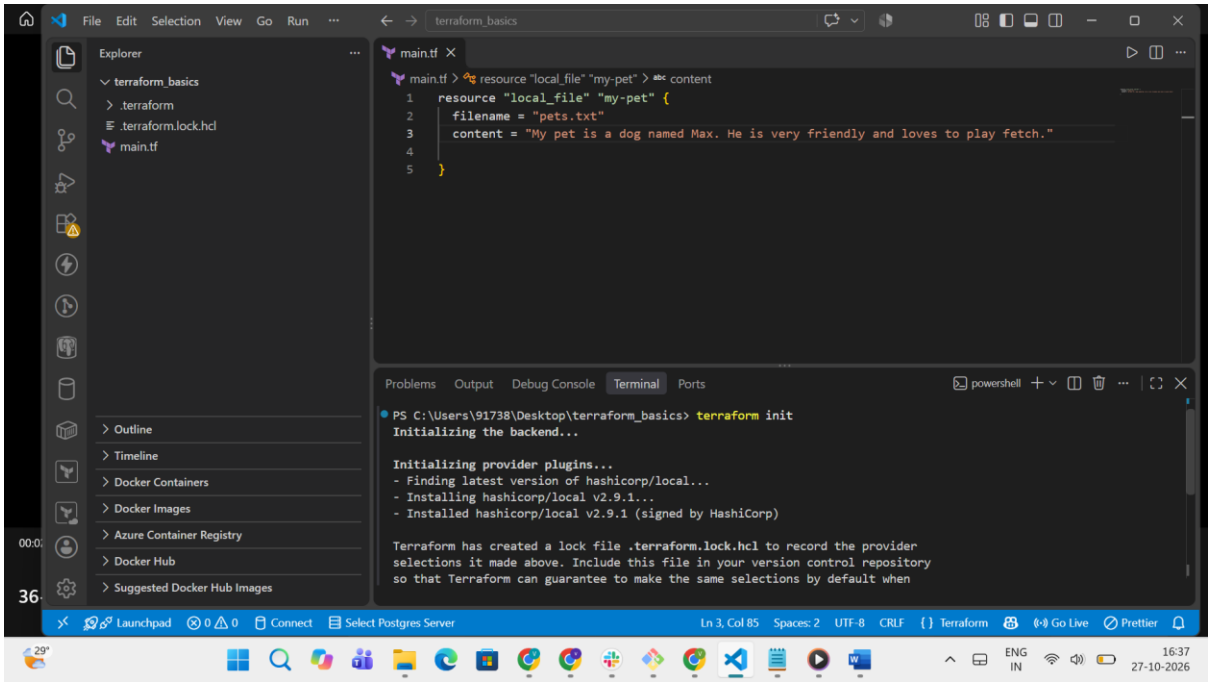
version = "2.3.0" --> download the exact version

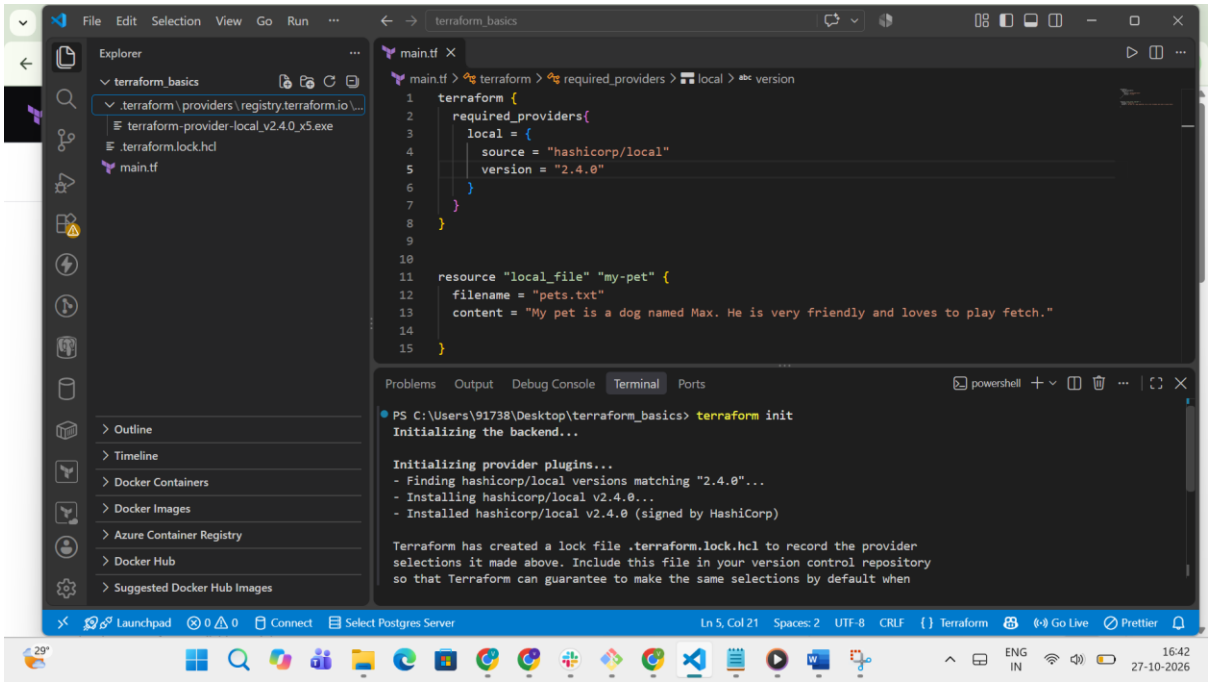
version = "!=2.3.0" --> will not use the mentioned version

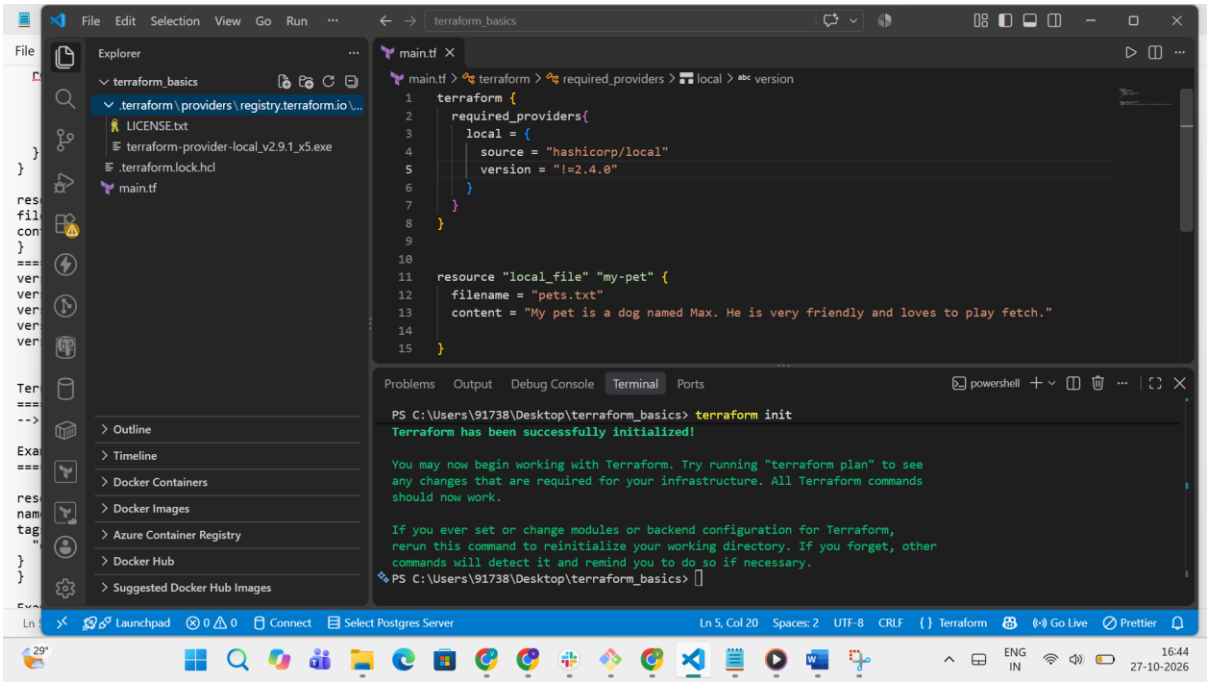
version = "< 2.3.0" --> lesses than the mention version

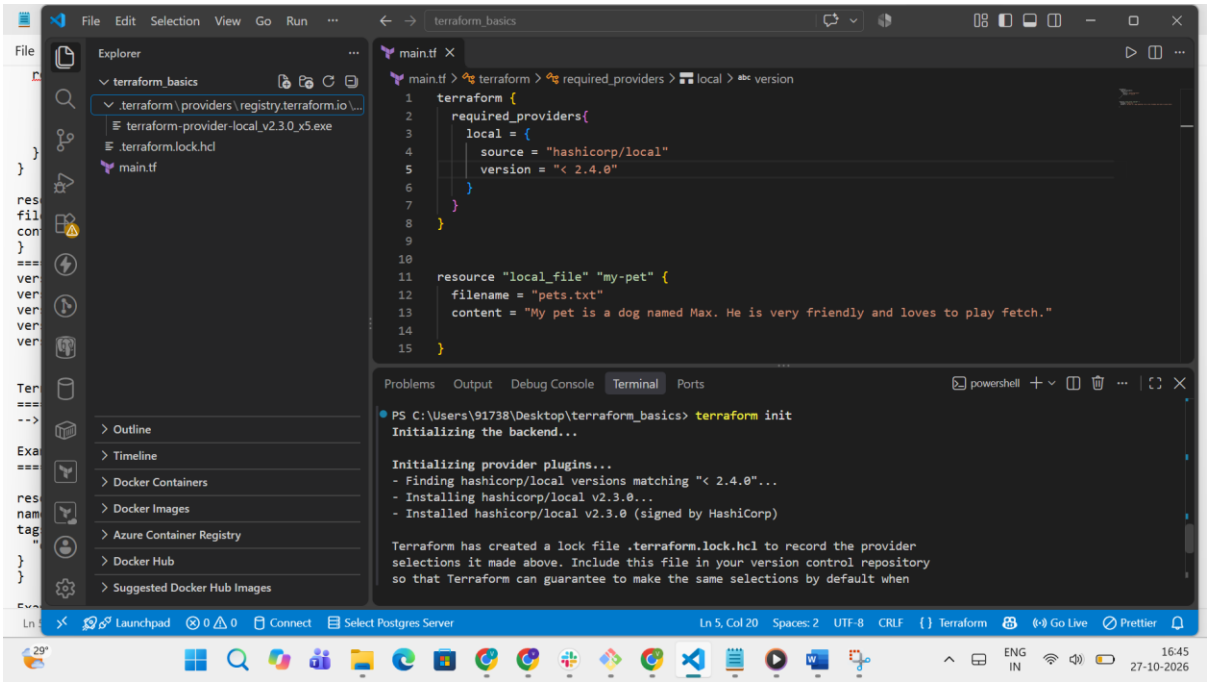
version = "> 2.3.0" --> greater than the given version

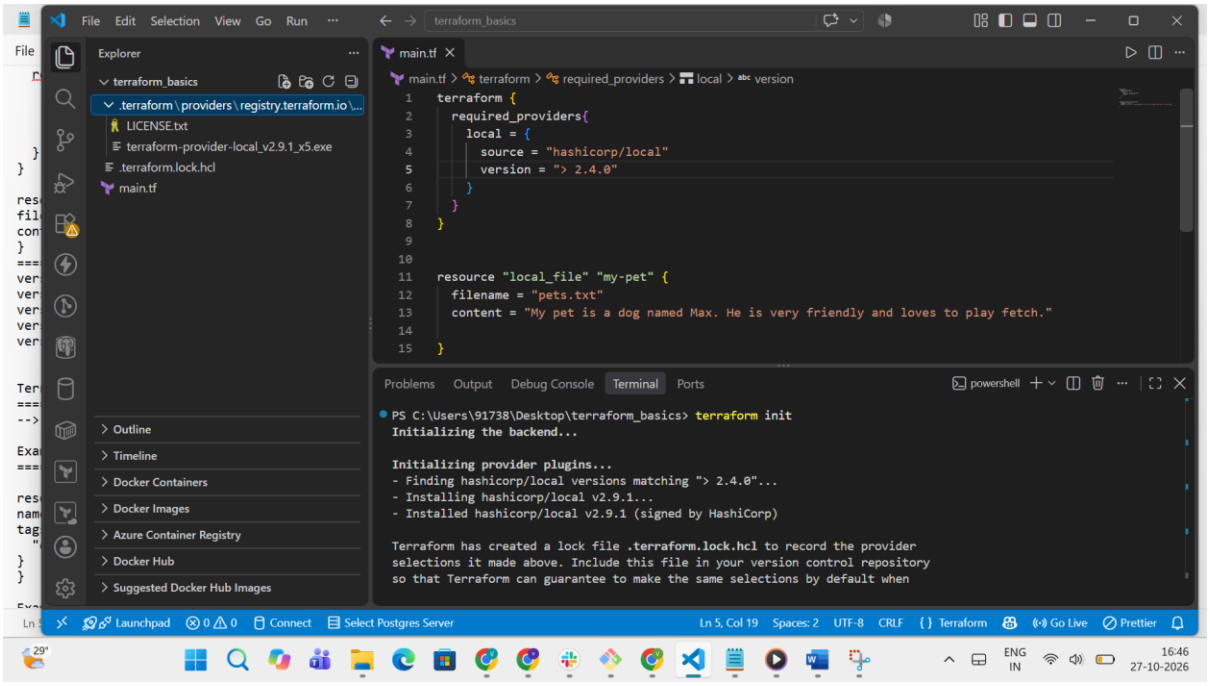
version = "~> 2.3.0" --> specific version or higher version.

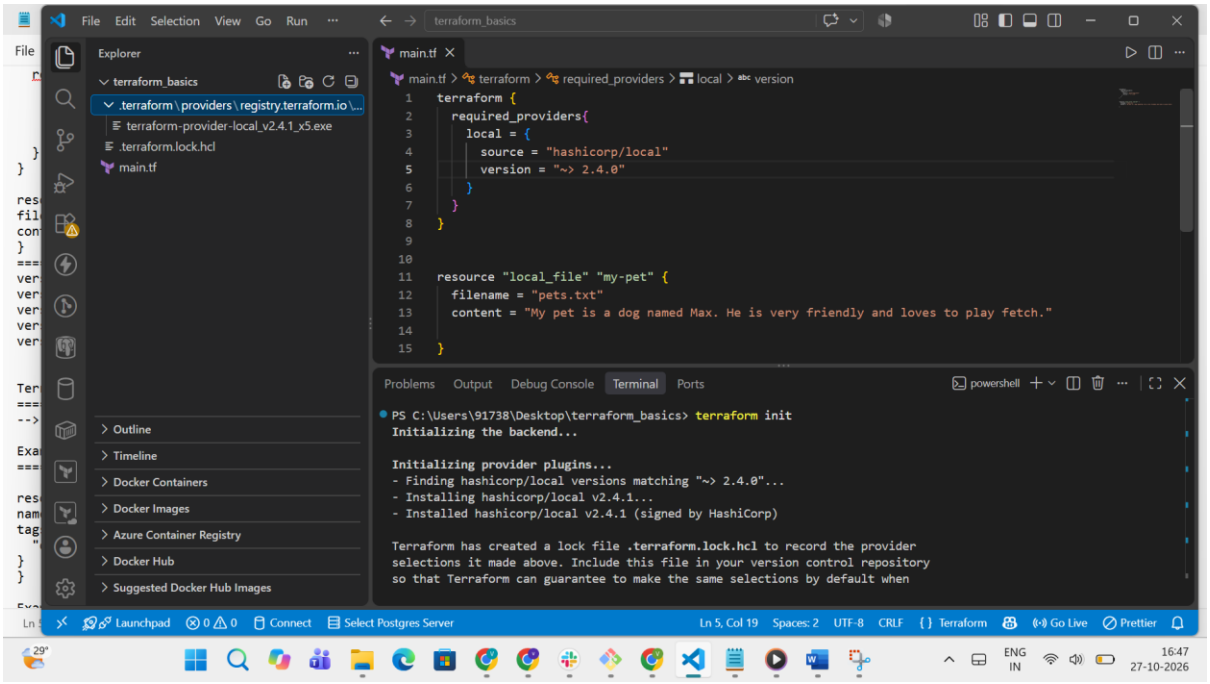





File Edit Selection View Go Run
terraform_basics
Explorer
terraform_basics
.terraform \ providers \ registry.terraform.io \...
LICENSE.txt
terraform-provider-local_v2.9.1_x5.exe
.terraform.lock.hcl
main.tf
main.tf
main.tf > terraform > required_providers > local > version
terraform {
required_providers{
local = {
source = "hashicorp/local"
version = "!=2.4.0"
}
}
}
resource "local_file" "my-pet" {
filename = "pets.txt"
content = "My pet is a dog named Max. He is very friendly and loves to play fetch."
}
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\91738\Desktop\terraform_basics> terraform init
Terraform has been successfully initialized!
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\91738\Desktop\terraform_basics>
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images
Launchpad
Connect
Select Postgres Server
Ln 5, Col 20
Spaces: 2
UTF-8
CRLF
Terraform
Go Live
Prettier
ENG
IN
16:44
27-10-2026


File Edit Selection View Go Run
terraform_basics
Explorer
terraform_basics
.terraform \ providers \ registry.terraform.io \...
terraform-provider-local_v2.3.0_x5.exe
.terraform.lock.hcl
main.tf
main.tf
main.tf > terraform > required_providers > local > version
terraform {
required_providers{
local = {
source = "hashicorp/local"
version = "< 2.4.0"
}
}
}
resource "local_file" "my-pet" {
filename = "pets.txt"
content = "My pet is a dog named Max. He is very friendly and loves to play fetch."
}
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\91738\Desktop\terraform_basics> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/local versions matching "< 2.4.0"...
- Installing hashicorp/local v2.3.0...
- Installed hashicorp/local v2.3.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images
Launchpad
Connect
Select Postgres Server
Ln 5, Col 20
Spaces: 2
UTF-8
CRLF
Terraform
Go Live
Prettier
ENG
IN
16:45
27-10-2026





main.tf

======

resource "local_file" "my-pet" {

filename = "pets.txt"

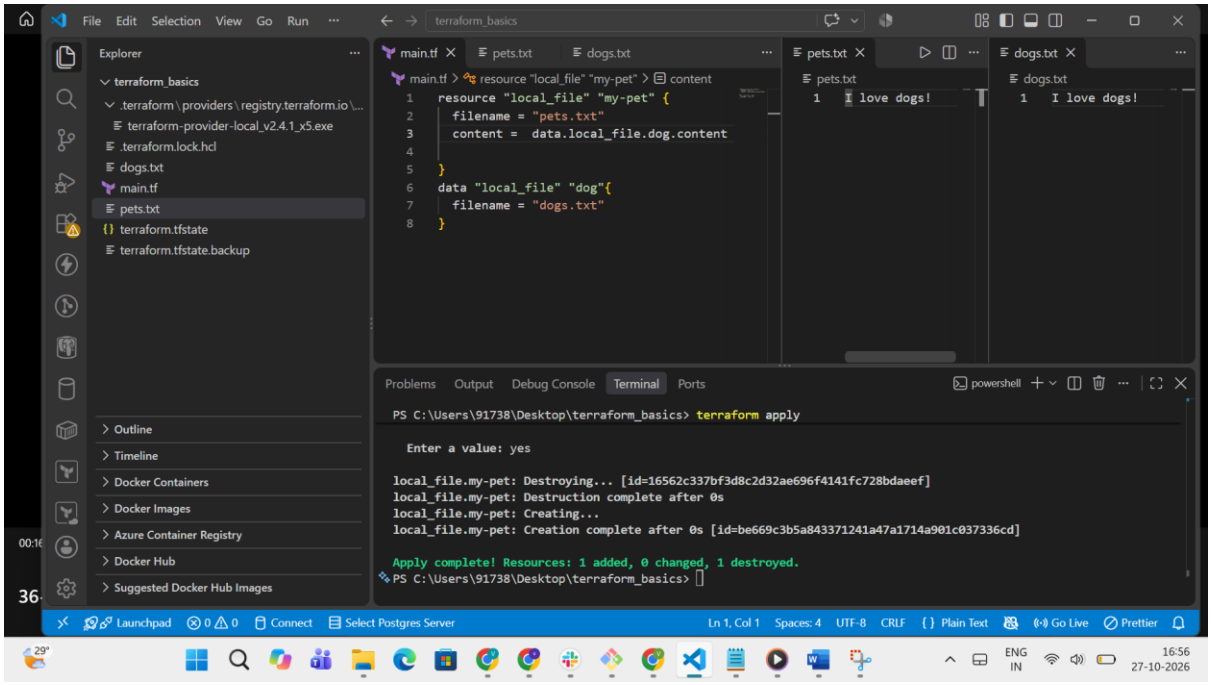
content = data.local_file.dog.content

}

data "local_file" "dog" {

filename = "dogs.txt"

}



resource "local_file" "my-pet" {

filename = var.filename[count.index]

content = "I love cats!"

count = lenght(var.filename)

}

variables.tf

variable "filename" {

default = [

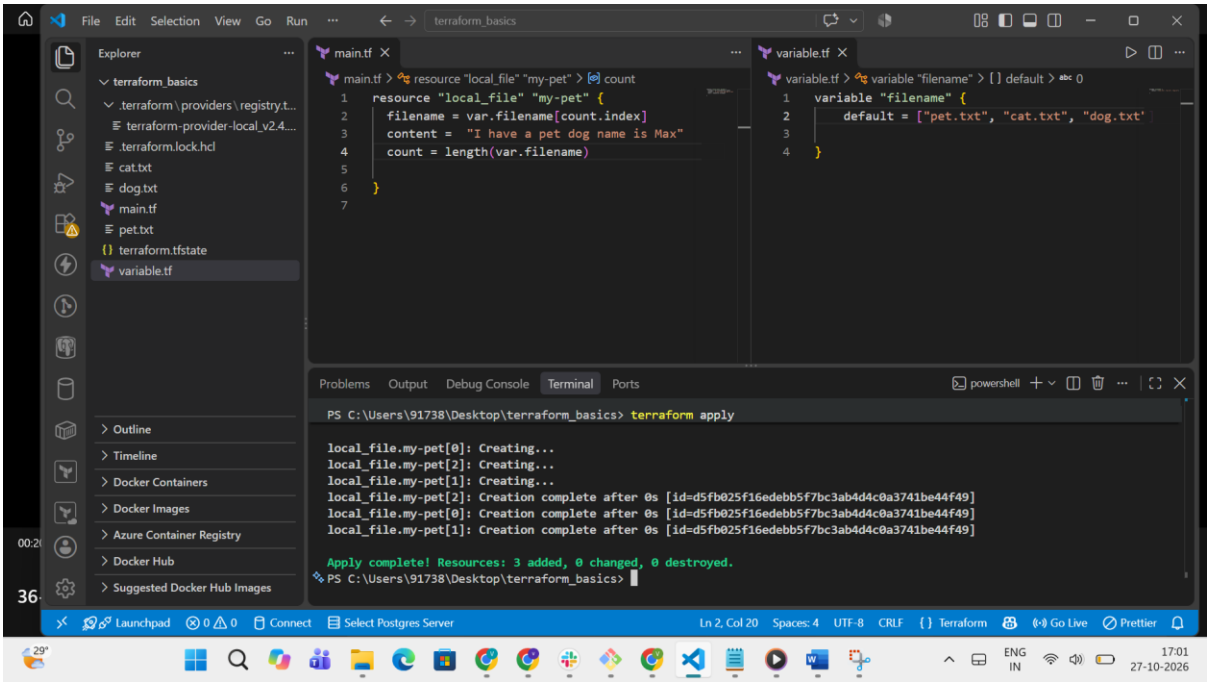
"pets.txt"

"cats.txt"

"dogs.txt"

]

}



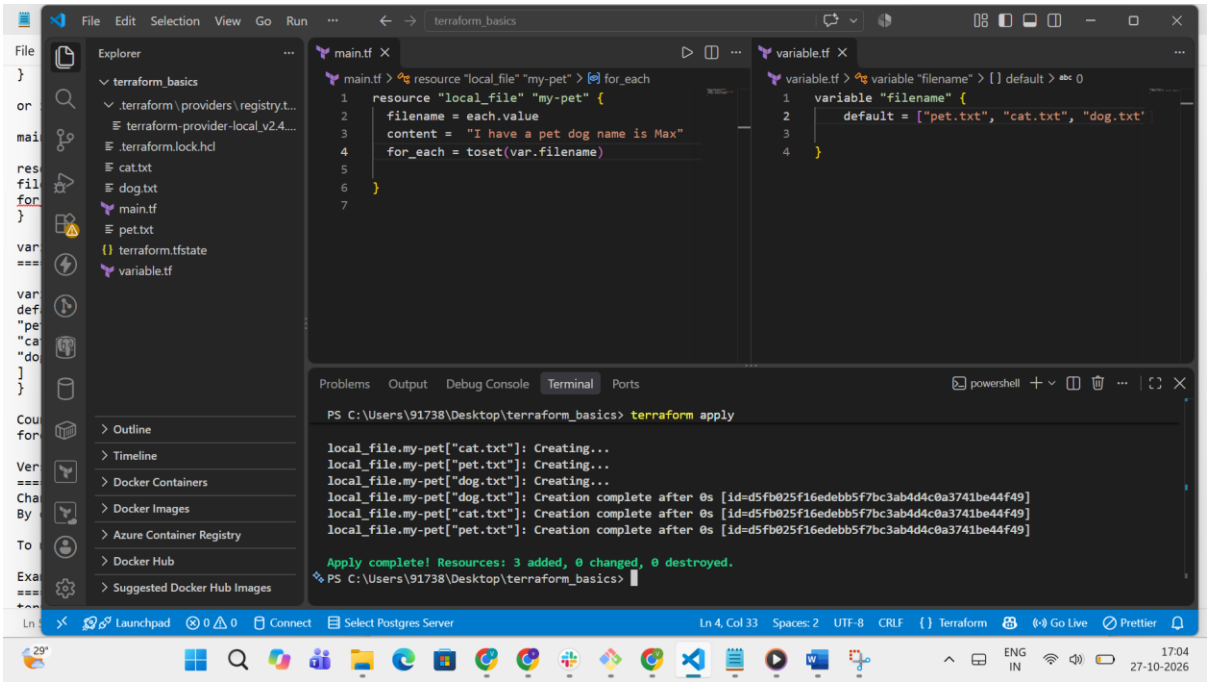
main.tf:

```
resource "local_file" "pet" {
filename = each.value
for_each = toset(var.filename)
}
```

variables.tf

==========

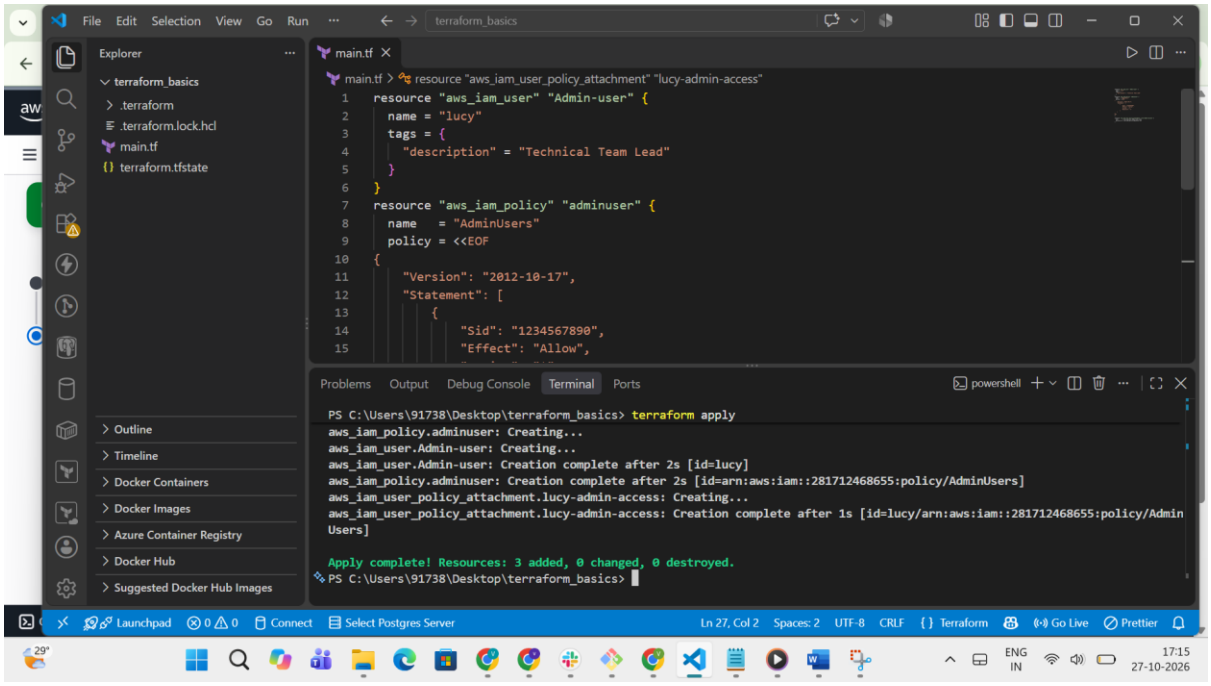
```
variable "filename" {
default = [
"pets.txt"
"cats.txt"
"dogs.txt"
]
}
```

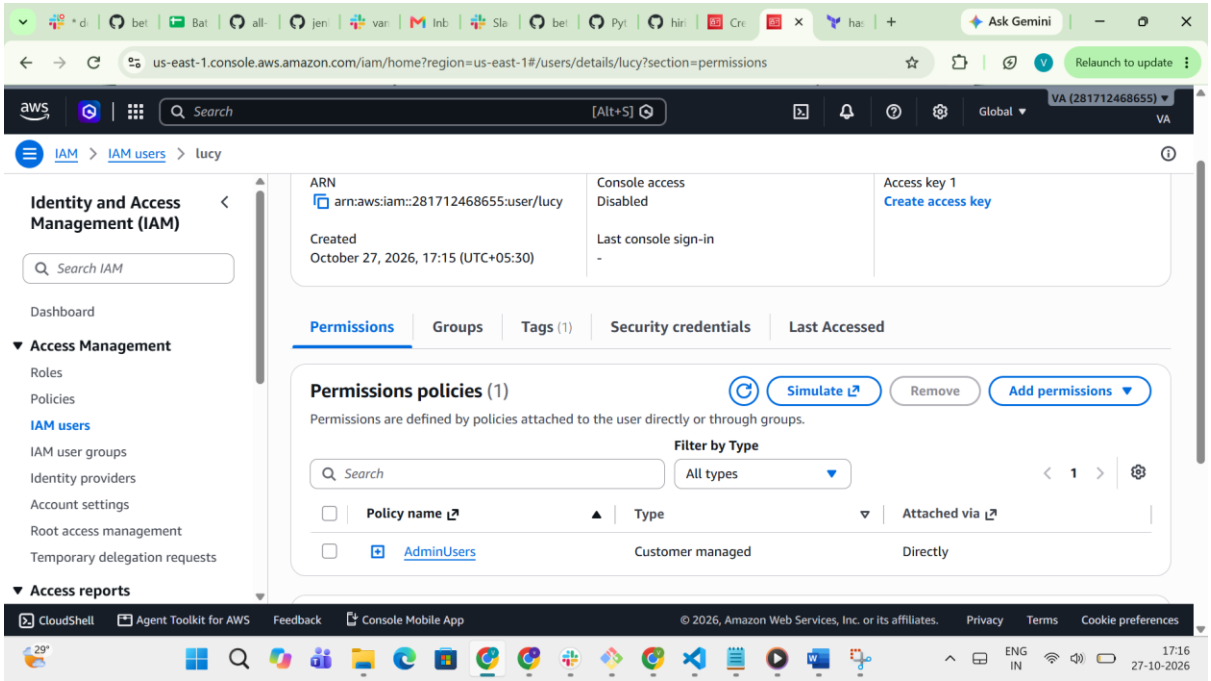



```
resource "aws_iam_user" "Admin-user" {
 name = "lucy"
 tags = {
  "description" = "Technical Team Lead"
 }
}
resource "aws_iam_policy" "adminuser" {
 name   = "AdminUsers"
 policy = <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "1234567890",
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*"
    }
  ]
```

```
}
EOF
}

resource "aws_iam_user_policy_attachment" "lucy-admin-access" {
  user       = aws_iam_user.Admin-user.name
  policy_arn = aws_iam_policy.adminuser.arn
}
```

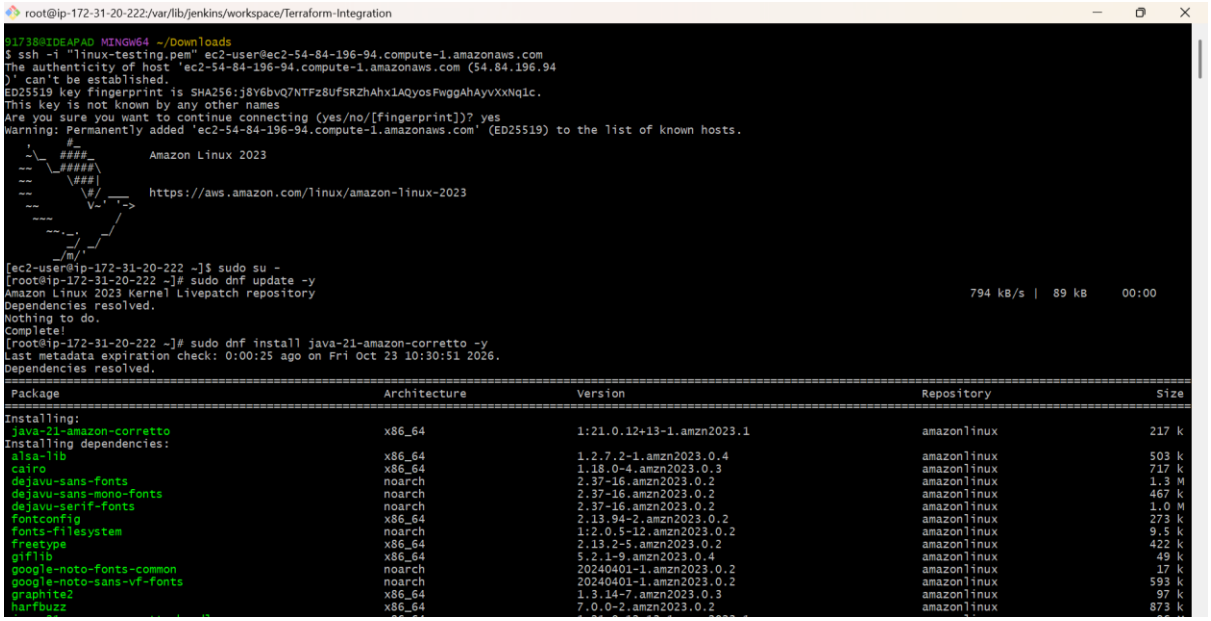


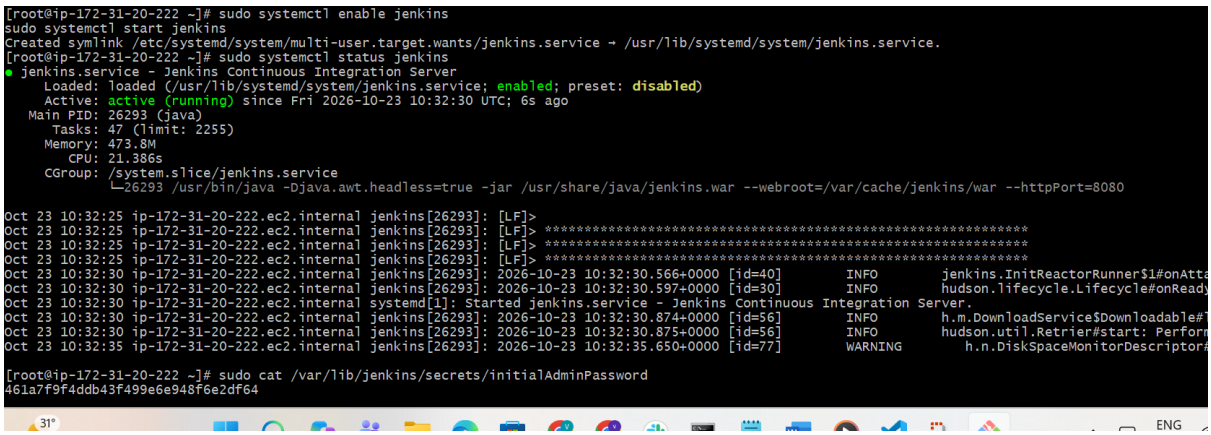


3.Integrate Terraform with Jenkins Using the Terraform Plugin

Implementation steps:

1. Logged in to the Jenkins dashboard.
2. Navigated to Manage Jenkins → Plugins.
3. Installed the Terraform plugin.
4. Navigated to Manage Jenkins → Tools.
5. Configured the Terraform installation and specified the Terraform executable path.
6. Saved the tool configuration.



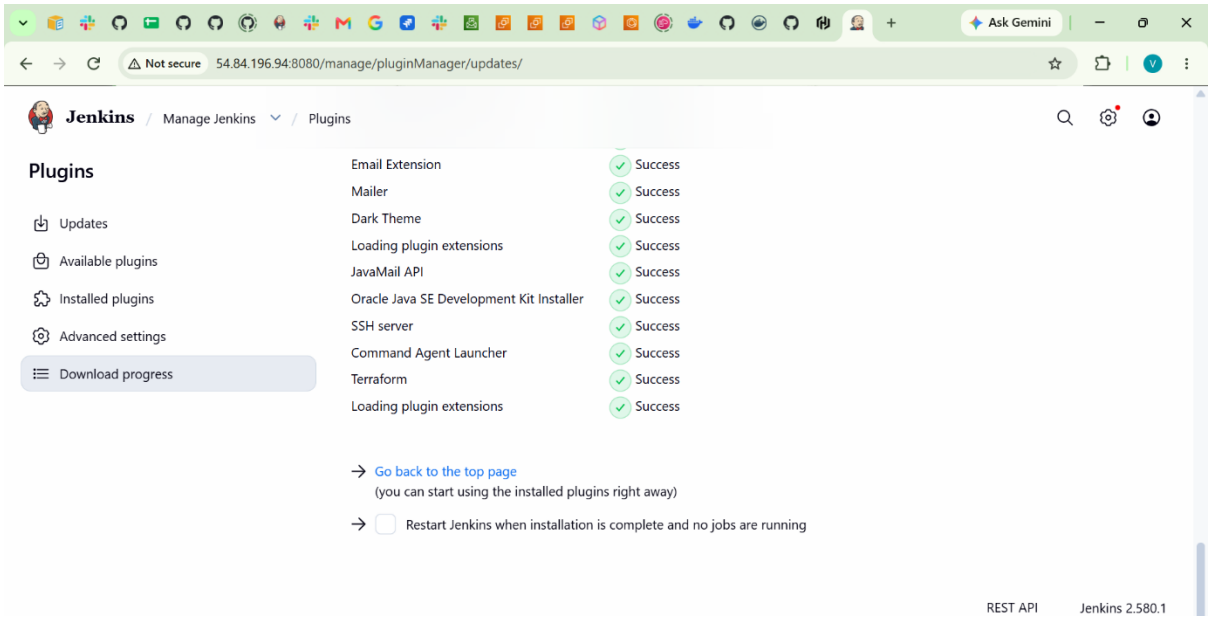


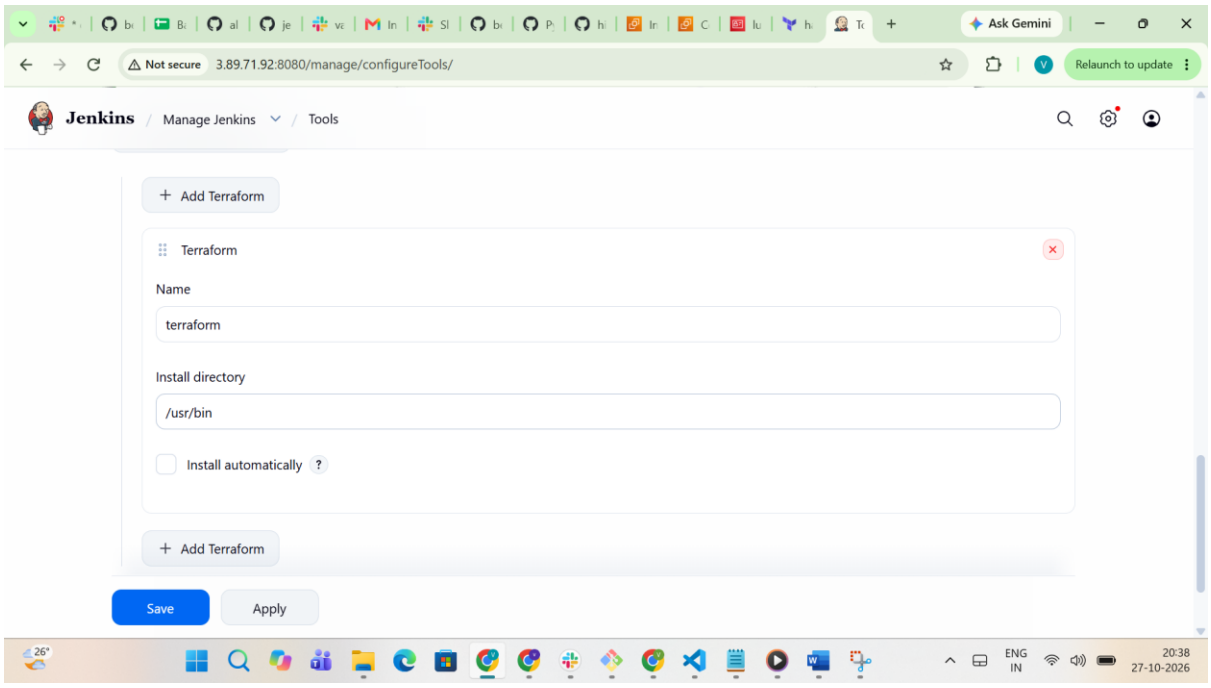
```
Installed:
  git-2.50.1-1.amzn2023.0.1.x86_64              git-core-2.50.1-1.amzn2023.0.1.x86_64 git-core-doc-2.50.1-1.amzn2023.0.1.noarch   perl-Error-1:0.17030-2.amzn2023.0.1.noa
  perl-File-Find-1.37-477.amzn2023.0.10.noarch perl-Git-2.50.1-1.amzn2023.0.1.noarch perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64 perl-lib-0.65-477.amzn2023.0.10.x86_64
  terraform-1.17.0-1.x86_64

Complete!
[root@ip-172-31-20-222 ~]# terraform version
Terraform v1.17.0
on linux_amd64

[root@ip-172-31-20-222 ~]# which terraform
/usr/bin/terraform
[root@ip-172-31-20-222 ~]# sudo -u jenkins terraform version
Terraform v1.17.0
on linux_amd64

[root@ip-172-31-20-222 ~]# readlink -f "$(which terraform)"
/usr/bin/terraform
[root@ip-172-31-20-222 ~]# dirname "$(readlink -f "$(which terraform)")"
/usr/bin
[root@ip-172-31-20-222 ~]# ls -l /opt/terraform
ls: cannot access '/opt/terraform': No such file or directory
[root@ip-172-31-20-222 ~]# ^C
[root@ip-172-31-20-222 ~]# cd /usr
[root@ip-172-31-20-222 usr]# ll
total 176
dr-xr-xr-x.  2 root root 32768 Oct 23 10:44 bin
drwxr-xr-x.  2 root root     6 Jan 30  2023 games
drwxr-xr-x.  3 root root    23 Oct 21 17:47 include
dr-xr-xr-x. 32 root root 16384 Oct 23 10:31 lib
dr-xr-xr-x. 44 root root 32768 Oct 23 10:31 lib64
drwxr-xr-x. 27 root root 16384 Oct 23 10:44 libexec
drwxr-xr-x. 12 root root   131 Oct 21 17:47 local
dr-xr-xr-x.  2 root root 16384 Oct 21 17:49 sbin
drwxr-xr-x. 98 root root 16384 Oct 23 10:44 share
drwxr-xr-x.  4 root root    34 Oct 21 17:47 src
lrwxrwxrwx.  1 root root    10 Jan 30  2023 tmp -> ../var/tmp
[root@ip-172-31-20-222 usr]# ^C
[root@ip-172-31-20-222 usr]# sudo -u jenkins terraform version
Terraform v1.17.0
on linux_amd64
```





4.Create a CI/CD Pipeline for the Node.js Application

Repository: https://github.com/betawins/Trading-UI.git

Implementation steps:

1. Created a Jenkins Pipeline job named Trading-ui-pipeline.

2. Connected the pipeline to the GitHub repository https://github.com/betawins/Trading-UI.git.

3. Configured the pipeline to check out source code from the master branch.

4. Verified that Node.js, npm, and Git were installed on the Jenkins server.

5. Installed the project dependencies using npm install --legacy-peer-deps to handle legacy peer dependency conflicts.

6. Installed ESLint version 6.8.0 temporarily to troubleshoot the dependency compatibility issue without modifying the original repository.

7. Executed the React production build using npm run build.

8. Resolved the OpenSSL compatibility error by adding NODE_OPTIONS=--openssl-legacy-provider to the build command.

9. Encountered ESLint warnings related to unused variables, unused imports, and invalid JSX scope properties.

10. Updated the temporary pipeline build command to use CI=false so ESLint warnings would not automatically fail the test build.

11. Added an artifact archiving stage to save the generated production files from the build/ directory.

12. Prepared to rerun the pipeline and verify the build and artifact archiving results.

2. Jenkins Pipeline code

Use this code in Jenkins → Trading-ui-pipeline → Configure → Pipeline → Definition: Pipeline script.

```
pipeline {
  agent any


  stages {
    stage('Checkout') {
      steps {
        git branch: 'master',
          url: 'https://github.com/betawins/Trading-UI.git'
      }
```

```
        }

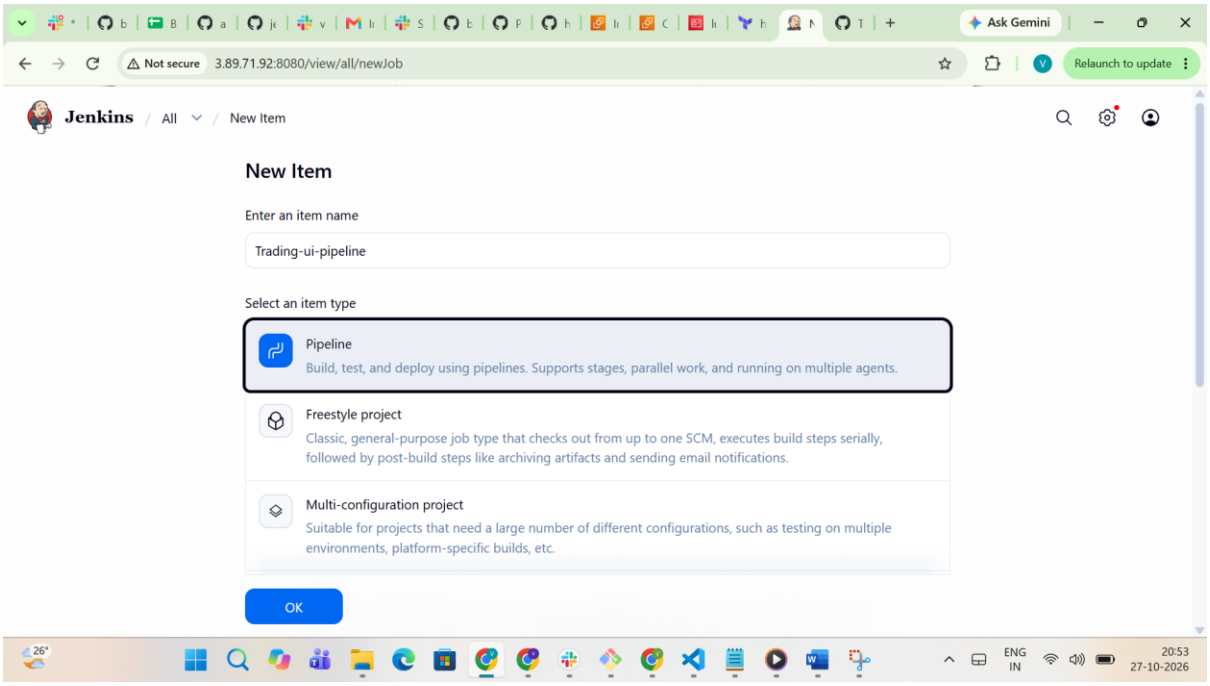
        stage('Install Dependencies') {
            steps {
                sh 'npm install --legacy-peer-deps'
                sh 'npm install --no-save --legacy-peer-deps eslint@6.8.0'
            }
        }

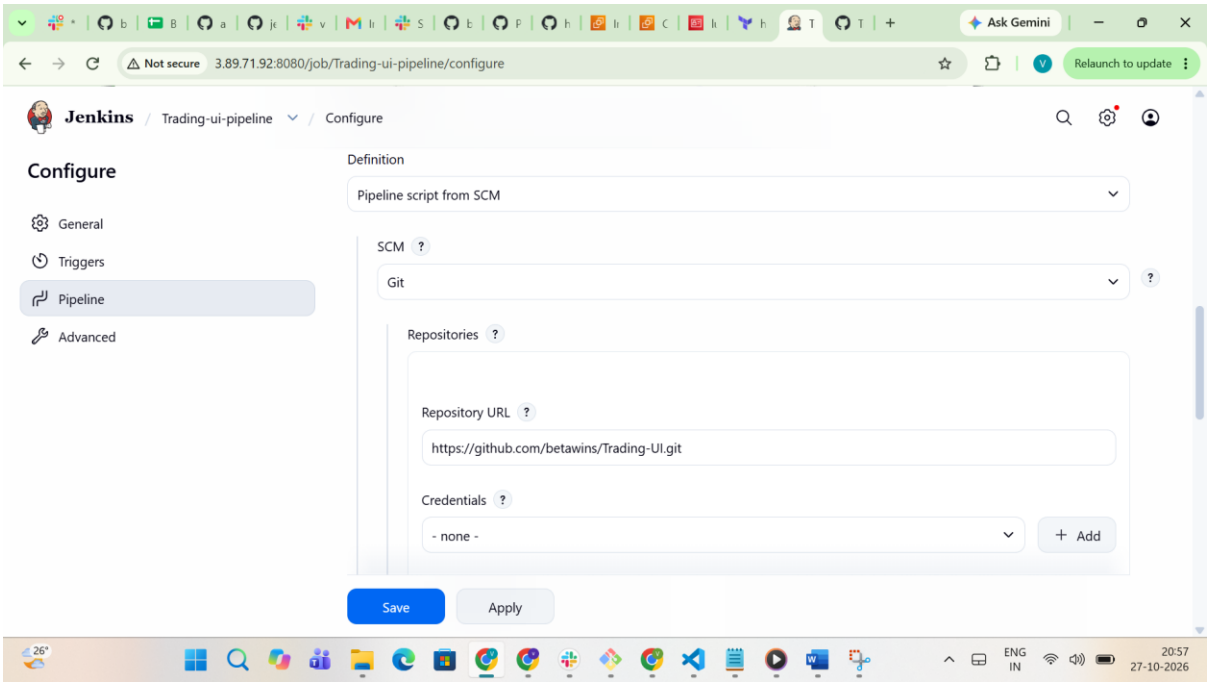
        stage('Build') {
            steps {
                sh 'CI=false NODE_OPTIONS=--openssl-legacy-provider npm run build'
            }
        }

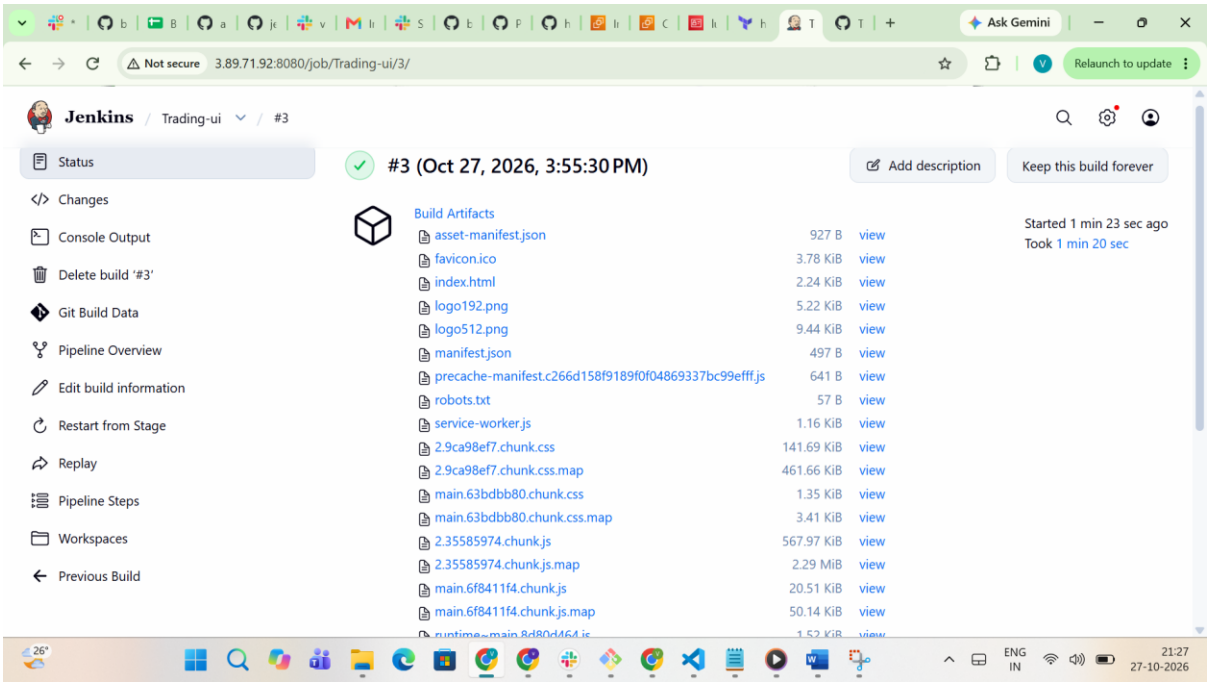
        stage('Archive Build') {
            steps {
                archiveArtifacts artifacts: 'build/**',
                                 allowEmptyArchive: false
            }
        }
    }
}
```

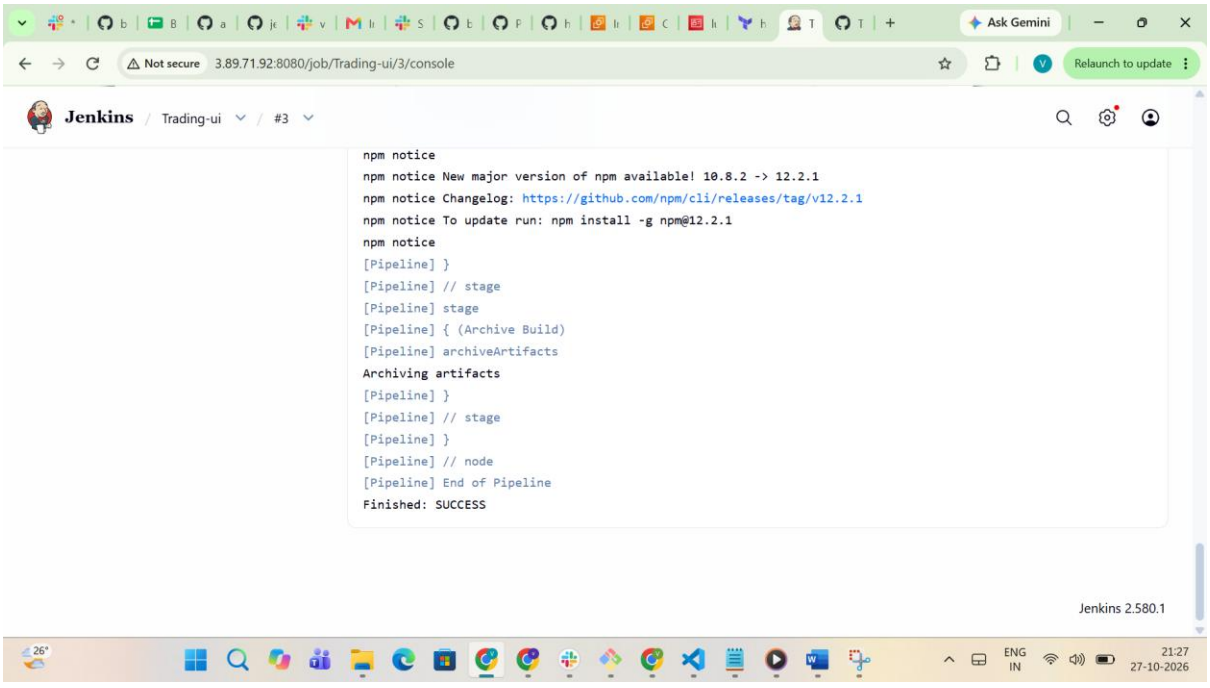
3. Execution procedure

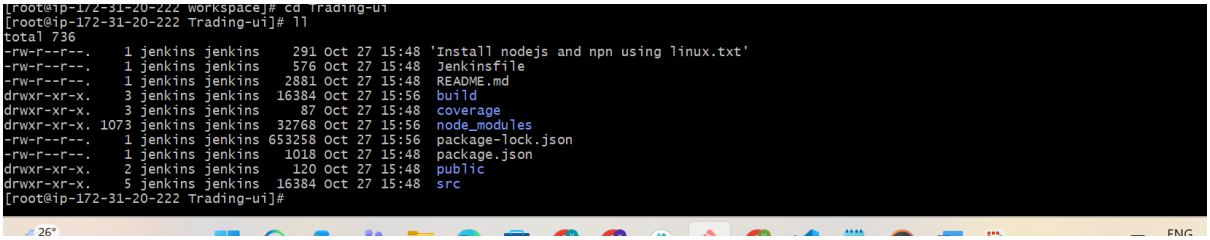
1. Open the Jenkins dashboard.
2. Select Trading-ui-pipeline and click Configure.
3. Select Pipeline script as the definition.
4. Paste the pipeline code and click Save.
5. Click Build Now.
6. Open Console Output to monitor each stage.
7. Verify that the build succeeds and the build/ directory is archived.


Not secure
3.89.71.92:8080/view/all/newJob
Relaunch to update
Jenkins
All
New Item
New Item
Enter an item name
Trading-ui-pipeline
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
OK


Not secure
3.89.71.92:8080/job/Trading-ui-pipeline/configure
Relaunch to update
Jenkins
Trading-ui-pipeline
Configure
Configure
General
Triggers
Pipeline
Advanced
Definition
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/betawins/Trading-UI.git
Credentials
- none -
Add
Save
Apply


Not secure 3.89.71.92:8080/job/Trading-ui/3/
Relaunch to update
Jenkins / Trading-ui / #3
Status
Changes
Console Output
Delete build '#3'
Git Build Data
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
#3 (Oct 27, 2026, 3:55:30 PM)
Add description
Keep this build forever
Build Artifacts
asset-manifest.json 927 B view
favicon.ico 3.78 KiB view
index.html 2.24 KiB view
logo192.png 5.22 KiB view
logo512.png 9.44 KiB view
manifest.json 497 B view
precache-manifest.c266d158f9189f0f04869337bc99efff.js 641 B view
robots.txt 57 B view
service-worker.js 1.16 KiB view
2.9ca98ef7.chunk.css 141.69 KiB view
2.9ca98ef7.chunk.css.map 461.66 KiB view
main.63bdbb80.chunk.css 1.35 KiB view
main.63bdbb80.chunk.css.map 3.41 KiB view
2.35585974.chunk.js 567.97 KiB view
2.35585974.chunk.js.map 2.29 MiB view
main.6f8411f4.chunk.js 20.51 KiB view
main.6f8411f4.chunk.js.map 50.14 KiB view
Started 1 min 23 sec ago
Took 1 min 20 sec


Not secure 3.89.71.92:8080/job/Trading-ui/3/console
Relaunch to update
Jenkins / Trading-ui / #3
npm notice
npm notice New major version of npm available! 10.8.2 -> 12.2.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.2.1
npm notice To update run: npm install -g npm@12.2.1
npm notice
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Archive Build)
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
Jenkins 2.580.1


[root@ip-172-31-20-222 workspace]# cd Trading-ui
[root@ip-172-31-20-222 Trading-ui]# ll
total 736
-rw-r--r--. 1 jenkins jenkins 291 Oct 27 15:48 'Install nodejs and npn using linux.txt'
-rw-r--r--. 1 jenkins jenkins 576 Oct 27 15:48 Jenkinsfile
-rw-r--r--. 1 jenkins jenkins 2881 Oct 27 15:48 README.md
drwxr-xr-x. 3 jenkins jenkins 16384 Oct 27 15:56 build
drwxr-xr-x. 3 jenkins jenkins 87 Oct 27 15:48 coverage
drwxr-xr-x. 1073 jenkins jenkins 32768 Oct 27 15:56 node_modules
-rw-r--r--. 1 jenkins jenkins 653258 Oct 27 15:56 package-lock.json
-rw-r--r--. 1 jenkins jenkins 1018 Oct 27 15:48 package.json
drwxr-xr-x. 2 jenkins jenkins 120 Oct 27 15:48 public
drwxr-xr-x. 5 jenkins jenkins 16384 Oct 27 15:48 src
[root@ip-172-31-20-222 Trading-ui]#

5.Execute and Understand 10 Maven Commands

1. Check Maven version

- Command: **mvn -version**
- Displays the installed Maven version and Java environment.

2. Clean the project

- Command: **mvn clean**
- Removes the previous build output.

3. Validate the project

- Command: **mvn validate**
- Checks whether the project configuration is valid.

4. Compile the source code

- Command: **mvn compile**
- Compiles the application's source code.

5. Run unit tests

- Command: **mvn test**
- Executes the configured unit tests.

6. Package the application

- Command: **mvn package**
- Packages the application into a JAR or WAR file.

7. Verify the project

- Command: **mvn verify**
- Runs the verification steps defined in the Maven lifecycle.

8. Install the package

- Command: **mvn install**
- Installs the generated artifact into the local Maven repository.

9. Deploy the package

- Command: **mvn deploy**
- Publishes the artifact to the configured remote Maven repository.

10. Display the dependency tree

- Command: **mvn dependency:tree**
- Displays the project's dependencies and their relationships.